

Final Project Planning

Overall implementation notes:

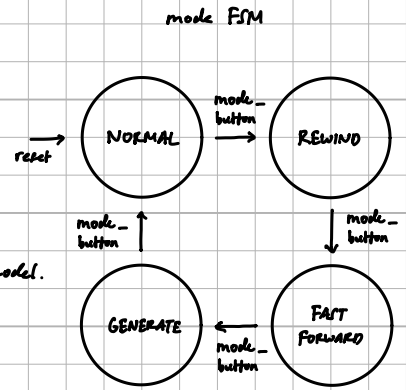
- we have configured an extra button to change "modes"

there are 4 modes: 1. NORMAL : plays stored songs forward, at normal speed.

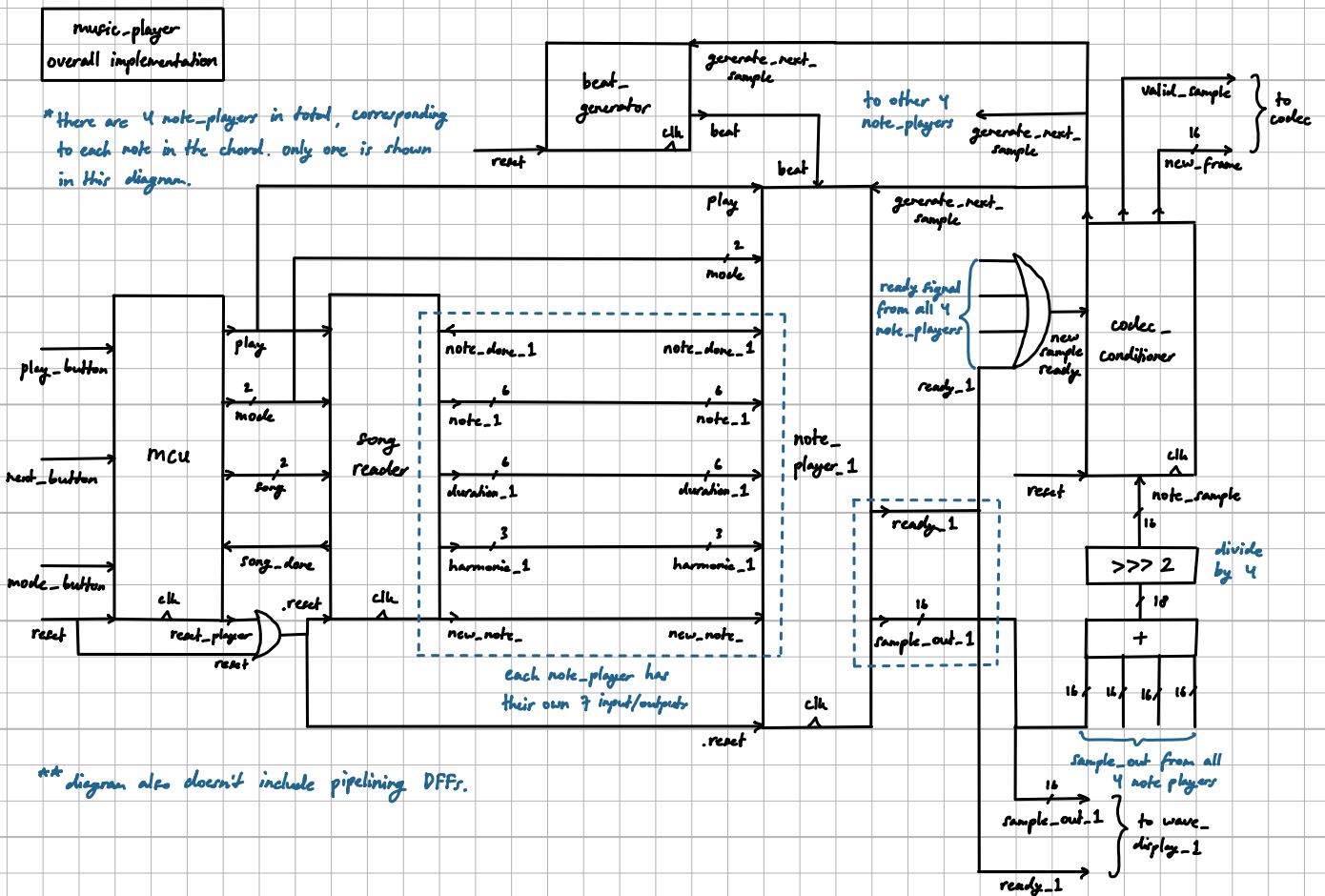
2. **REWIND** : plays stored songs in reverse, at normal speed.

3. **FAST_FORWARD** : plays stored songs forward, at double speed.

4. **GENERATE** : generates and plays a new song based on a neural network model.

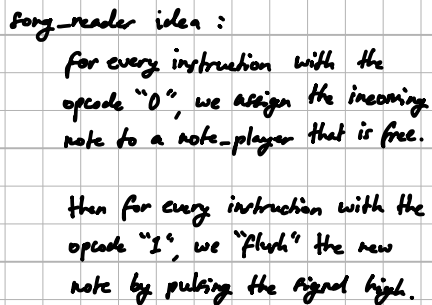
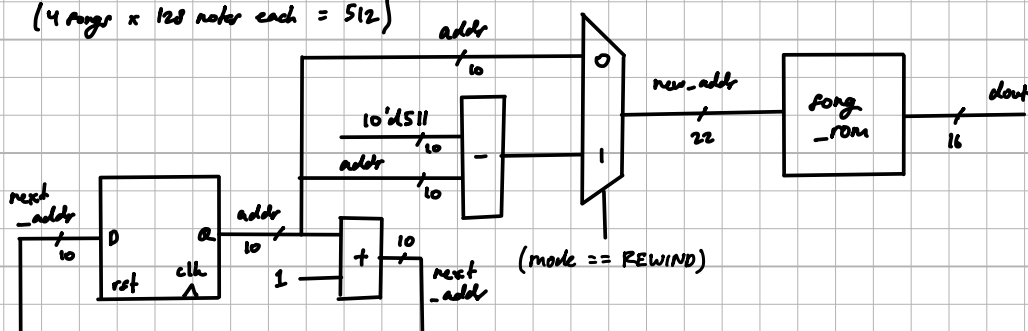


- we aim to implement 5 features:
 1. chords (4 points)
 2. playback control (4 points)
 3. harmoniser (3 points)
 4. enhanced waveform display (1 point)
 5. neural network melody generator (5 points)

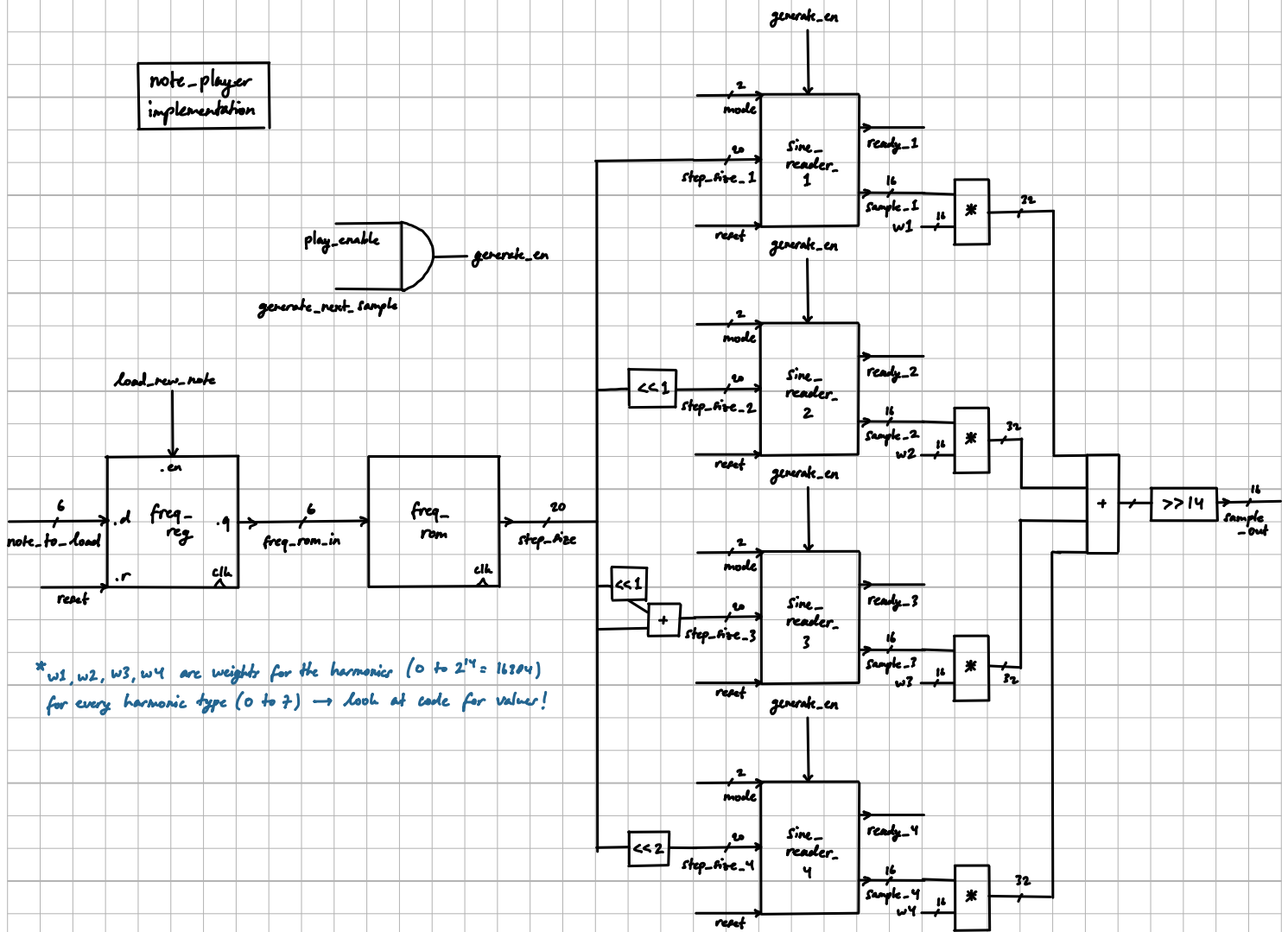


** diagram also doesn't include pipelining DFFs.

font_reader implementation

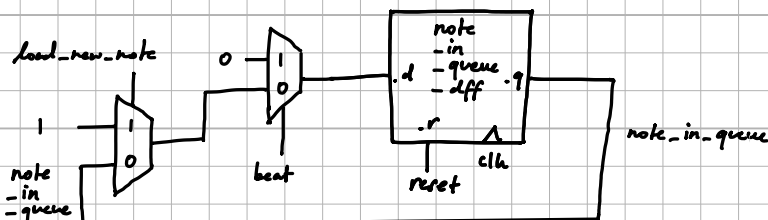
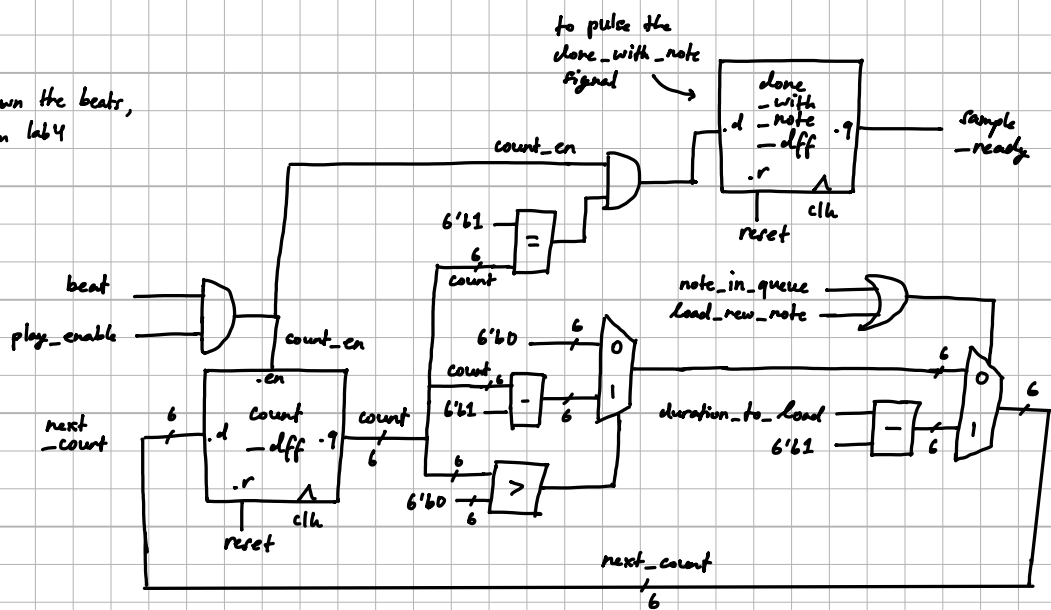

$$(4 \text{ rings} \times 128 \text{ notes each} = 512)$$


note_player
implementation



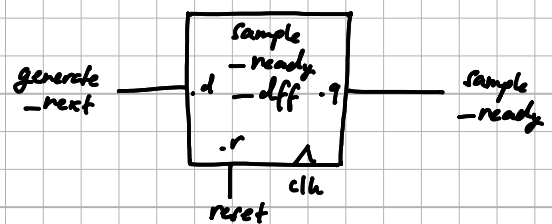
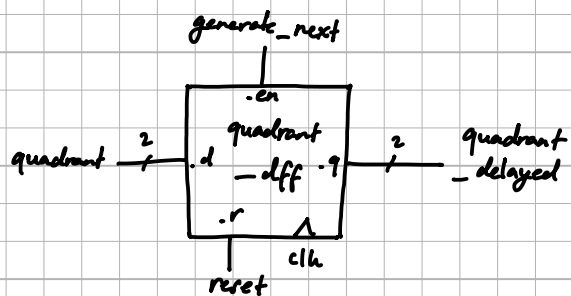
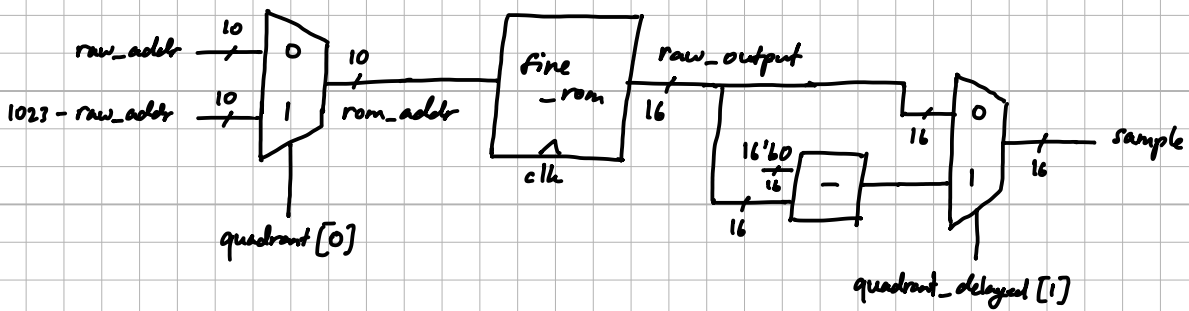
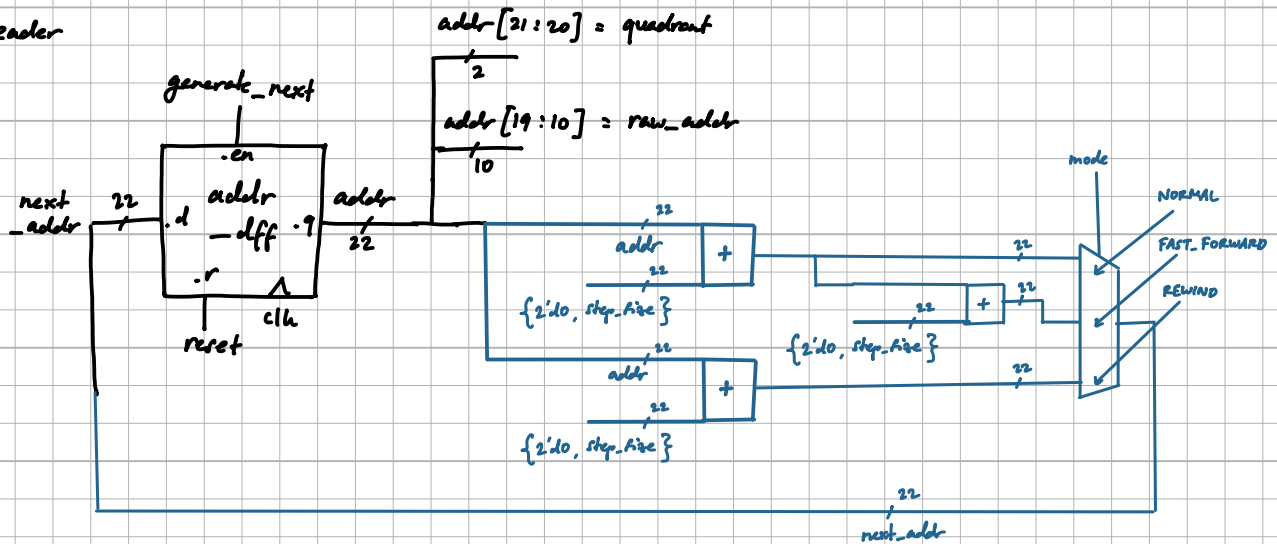
* w1, w2, w3, w4 are weights for the harmonics (0 to $2^{14} = 16384$)
for every harmonic type (0 to 7) → look at code for values!

to count down the beats,
→ from lab 4



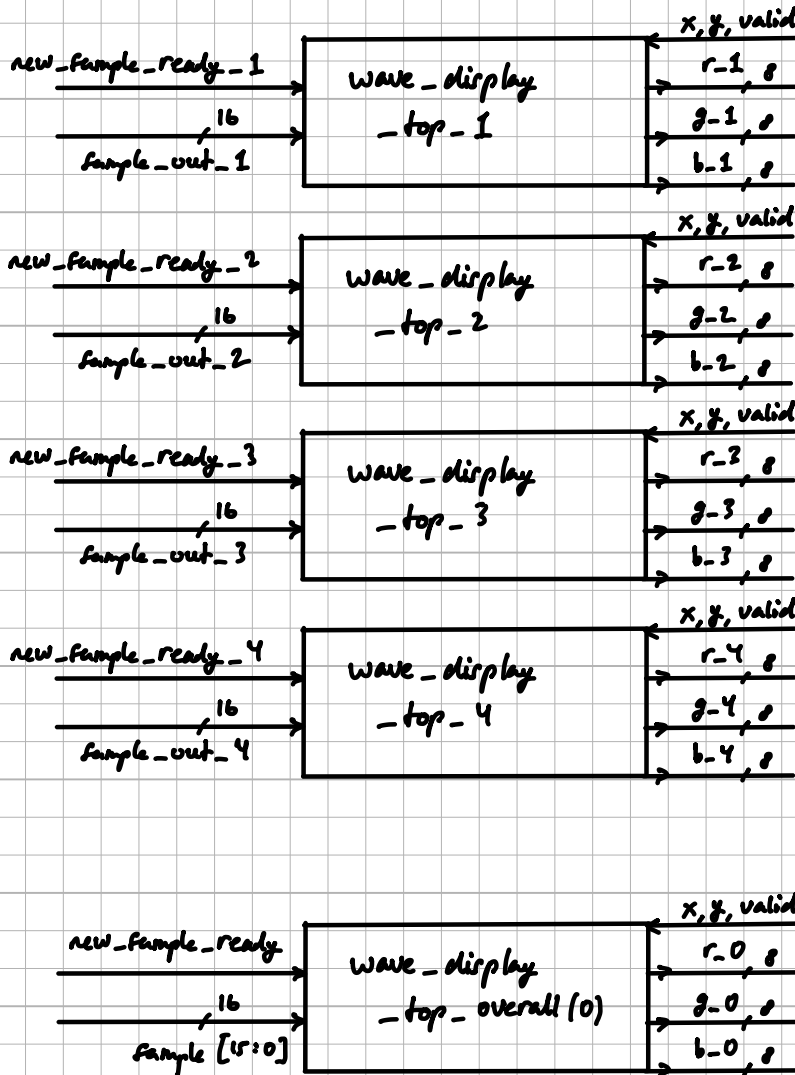
fine_reader implementation → from lab4-report

sine_reader



Enhanced Waveform Display: each of the 4 notes will be in a different color, with the combined waveform in white.

for each of the 5 total waveforms, we need to have a wave_display_top (lab5) module.



we assign `r-n`, `g-n`, `b-n` to a set value that is distinct from one another.

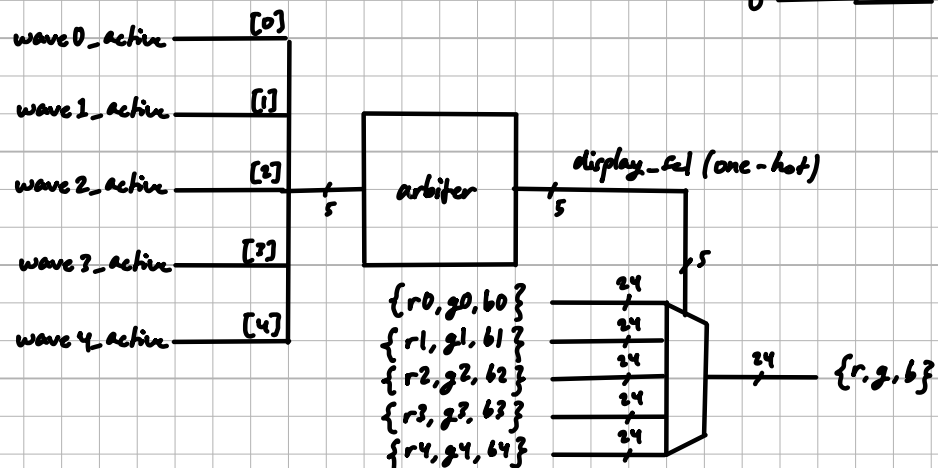
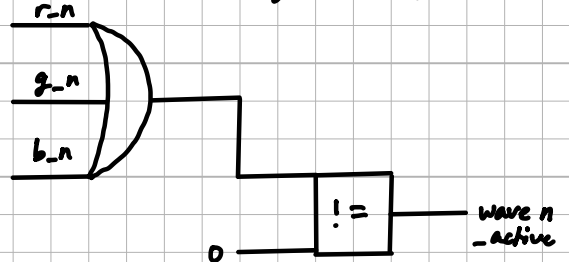
- 1 → blue
- 2 → red
- 3 → green
- 4 → pink
- overall → white

for every single color signal (`r`, `g`, `b`), we need to prioritize some signal to be put "on top" of others

↳ we prioritize 0 (overall) → 1 → 2 → 3 → 4

we can use a priority mux for this
↳ arbiter

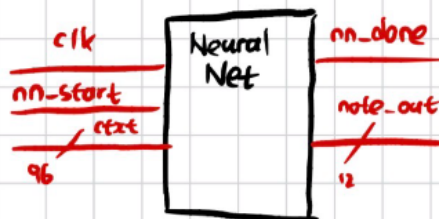
for each `wave_display_top`, (replace `n`)



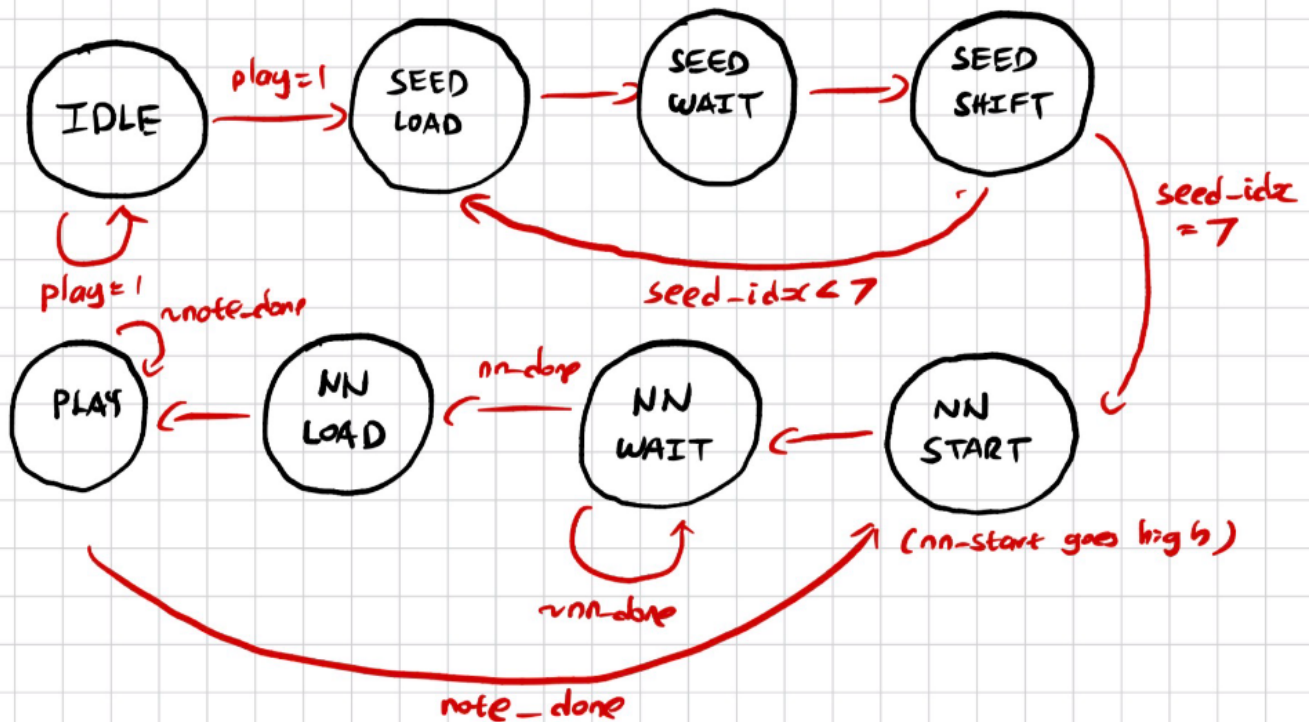
Neural Network Music Generator

Song Reader FSM

Idea: we can modify the FSM for song reader. Instead of having a WAIT state that waits for 1 cycle, it waits for a signal **nn_done** from the neural network module. before proceeding to the next state. Here is the interface for neural network module:



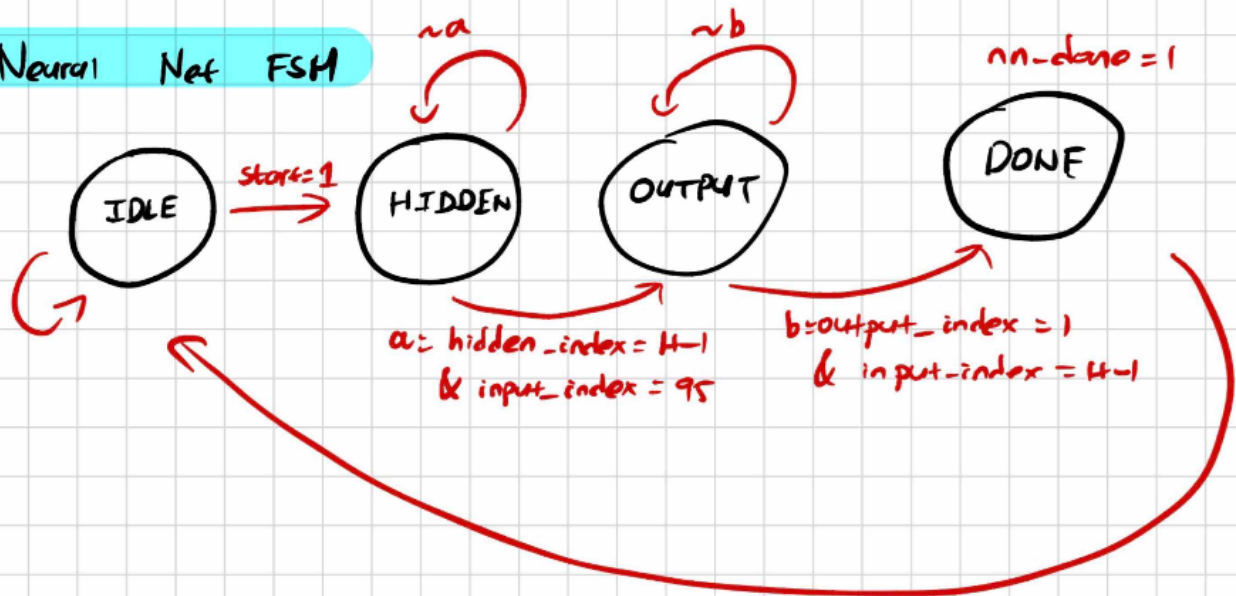
ctxt is a 96-bit signal representing the last 8 notes, with the LSBs representing the most outdated note. We also create a ROM to store the first 8 notes and durations ($8 \times 12 = 96$ bits) as context for the first note generation



Neural Net module (1 hidden layer + 1 output layer)

We will have Weight ROM with size $H \times 96$ (H is the hidden layer size) and the bias ROM (both are hardcoded)

Neural Net FSM



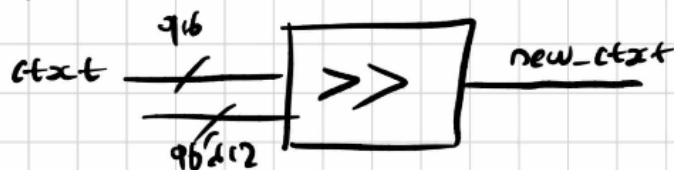
In hidden, given $x = \text{ctxt}[0 \dots 95]$, we do

$$\text{hidden_out}[h] = \text{ReLU} \left(\text{bias}[h] + \sum_{i=0}^{95} \text{weight_hidden}[h][i] \cdot \text{ctxt}[i] \right)$$

In output, we do :

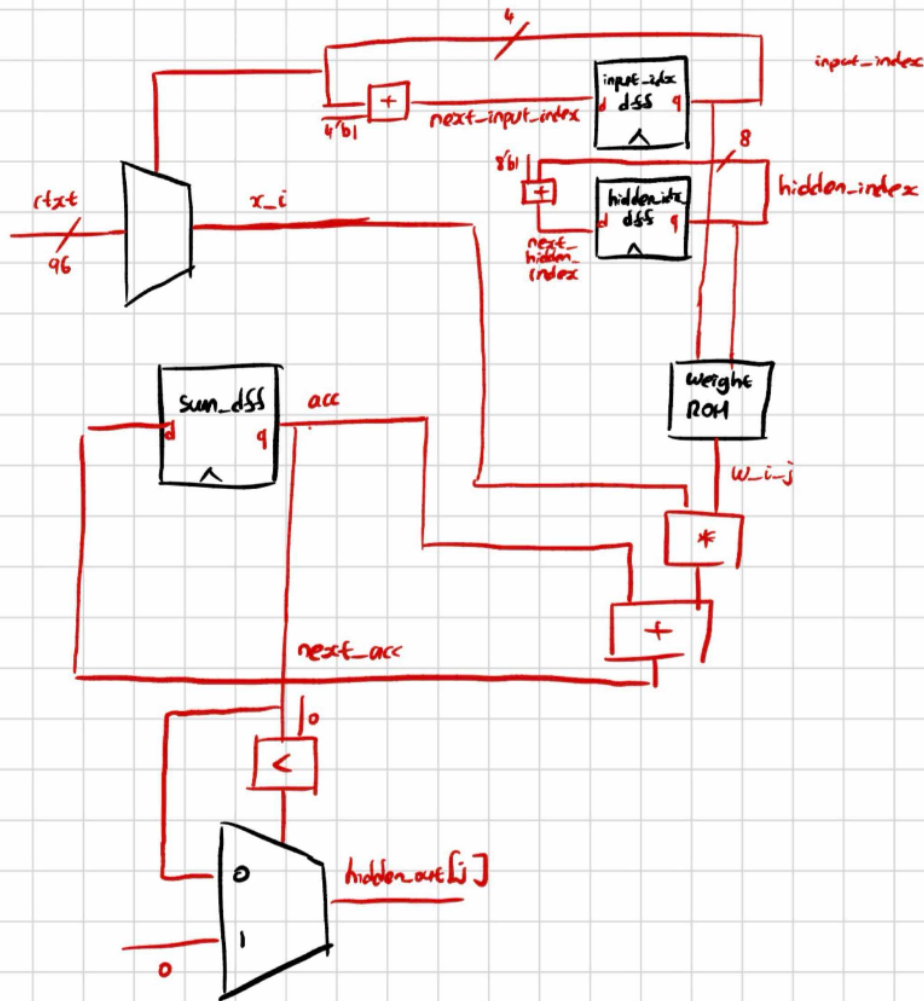
$$y[o] = \text{bias_out}[o] + \sum_{i=1}^{H-1} \text{weight_out}[o][i] \cdot \text{hidden_out}[i]$$

Given a new generated note new_note , this is how we update the context



$$\text{new_ctxt}[95:88] = \text{new_note}$$

Hidden Block



Output Block

